

Using FPU registers for temporary storage of data to reduce load/store instruction

Computer Architecture Project

November 23, 2025

1 Introduction

Modern processors provide several different storage locations for data: main memory, caches, and on-chip register files. Among the on-chip resources, floating-point unit (FPU) registers are extremely fast and are typically used to hold operands and results for floating-point operations. This document explores whether FPU registers can also be used as temporary storage for data that will be used in the immediate future, with the goal of reducing load/store traffic to memory and improving performance.

2 Background: Registers and Memory

2.1 General-Purpose Registers

General-purpose registers (GPRs) are integer registers used for:

- Arithmetic and logic on integer data.
- Address calculations for loads and stores.
- Holding pointers, loop indices, and general temporary values.

Accessing a value in a register typically takes one cycle in an in-order pipeline, whereas accessing data from memory requires additional cycles due to cache lookup, possible misses, and main memory latency.

2.2 Floating-Point Registers and the FPU

The floating-point unit (FPU) is a hardware block dedicated to:

- Addition, subtraction, multiplication, division on floating-point numbers.
- More complex operations such as square root or fused multiply-add, depending on the ISA.

The FPU has its own register file (floating-point registers). These registers are normally used to:

- Hold input operands for FP instructions.

- Store intermediate results and final results of FP computations.

The key observation is that FPU registers are on-chip, have low access latency, and can sometimes be repurposed as temporary storage for data that will be reused soon, potentially reducing memory accesses.

3 Architectural Overview

This section briefly summarizes general-purpose and FPU registers on three important architectures: ARM, Intel x86_64, and RISC-V.

3.1 ARM Architecture

On a typical 32-bit ARM (AArch32) or 64-bit ARMv8-A (AArch64) core:

- There are 16 or 31 general-purpose registers, depending on the execution state and ISA variant.
- With floating-point and SIMD extensions enabled (VFP/NEON), there is a separate FP/SIMD register file.

For ARMv8-A in AArch64 state:

- General-purpose registers: $x0-x30$ (64-bit).
- FP/SIMD registers: $v0-v31$, each of which can be viewed as multiple sizes (e.g., 32-bit, 64-bit, 128-bit).

The FP/SIMD registers can be used for:

- Scalar floating-point operations.
- Vector (SIMD) integer and floating-point operations.

This makes them capable of holding both FP data and packed integer data, which is useful for temporary storage in performance-critical loops.

3.2 Intel x86_64 Architecture

On modern Intel 64-bit processors:

- General-purpose registers: 16 registers (RAX, RBX, RCX, ..., R15), each 64 bits wide.
- Legacy x87 FPU: 8 floating-point registers organized as a stack ($ST(0)-ST(7)$).
- SIMD/FPU registers: $XMM0-XMM15$ (128-bit), and possibly YMM/ZMM for AVX/AVX-512.

In practice:

- The old x87 stack is less convenient for general temporary storage because of its stack-like semantics.
- SSE/AVX registers (XMM , YMM , ZMM) are flat, indexable register files that can hold scalar or vector FP and integer values, and thus are often used to keep temporary data in registers.

3.3 RISC-V Architecture

In a standard 64-bit RISC-V core with the floating-point extension:

- General-purpose registers: 32 registers $x0-x31$; $x0$ is hard-wired to zero.
- Floating-point registers: 32 registers $f0-f31$, whose width depends on the configured FP precision (e.g., 32-bit, 64-bit).

RISC-V has a clean separation between the integer register file and the FP register file. Load/store instructions exist for both integer and floating-point data, and FP registers are designed for efficient access by floating-point instructions.

4 Why Use FPU Registers for Temporary Storage?

4.1 Latency and Throughput

Register access latency is generally much lower than memory access latency. A simplified view:

- Register file access: often 1 cycle, sometimes slightly more depending on pipeline depth.
- L1 data cache access: a few cycles.
- L2/L3 cache or main memory: tens to hundreds of cycles in the worst case.

If a value is needed repeatedly in a tight loop, keeping it in a register (GPR or FPU register) avoids repeated memory load instructions and the corresponding cache/memory traffic.

4.2 Reducing Load/Store Instructions

Consider a loop performing floating-point operations:

- If the compiler or programmer stores intermediate results in memory, each use requires a load, and each update requires a store.
- If the same intermediate results are kept in FPU registers, the loop can avoid many of these loads and stores.

The idea of the project is to *intentionally* use FPU registers as an additional pool of fast storage to hold data that will be reused soon, thereby lowering the dynamic count of load/store instructions.

5 Potential Benefits

Using FPU registers for temporary storage can provide:

- **Lower latency:** Faster access compared to reading from memory .

- **Higher throughput:** Fewer memory operations can reduce pipeline stalls due to cache misses.
- **Better utilization of hardware:** On some workloads, the FPU register file may be underutilized while the integer register file is under pressure; using FP registers as extra temporaries can balance resource usage.

In computation-heavy kernels such as matrix multiplication, convolution, or other numerical algorithms, these advantages can translate into measurable performance improvements.

6 Potential Drawbacks and Limitations

Despite the potential benefits, there are important caveats.

6.1 Register Pressure and Spilling

Register files are limited in size:

- If too many values compete for registers, the compiler or programmer must spill some of them to memory.
- Using FPU registers for non-FP data can increase overall register pressure in FP-heavy code, causing more spills and negating the performance benefits.

The effective gain depends heavily on the specific kernel and how many values must be live at the same time.

6.2 Complexity and Portability

There are several complications:

- **Instruction set constraints:** Not all ISAs provide flexible data movement instructions between GPRs, FPU registers, and memory.
- **Calling conventions:** ABIs define which registers must be saved/restored across function calls; abusing FPU registers without respecting the ABI can lead to incorrect programs.
- **Portability:** Code that explicitly relies on FPU registers is often architecture-specific and may require different implementations for ARM, Intel, and RISC-V.

6.3 Context Switching and OS Overhead

When the OS performs a context switch:

- General-purpose registers must be saved and restored.
- FPU/SIMD state is also saved and restored, sometimes lazily.

If code heavily uses FPU registers as extra storage, the cost of saving/restoring this additional state may slightly increase context-switch overhead, which is relevant in very multitasked or real-time environments.

7 Example: C Code Using Memory vs. FPU Registers

7.1 Baseline C Code Using Memory

Below is a simple matrix multiplication kernel using a temporary variable `temp`, which the compiler may place on the stack or in a register depending on optimization:

```
void matrix_multiply_mem(float *A, float *B, float *C, int N) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            float temp = 0.0f;
            for (int k = 0; k < N; k++) {
                temp += A[i * N + k] * B[k * N + j];
            }
            C[i * N + j] = temp;
        }
    }
}
```

In a naive implementation or with low optimization, `temp` and intermediate values may be spilled to memory, incurring load/store overhead.

7.2 Conceptual Version Emphasizing FPU Registers (RISC-V Example)

On a RISC-V core with the floating-point extension, one can encourage keeping temporaries in FPU registers using inline assembly. The following is conceptual code showing accumulation directly in a floating-point register:

```
void matrix_multiply_fpu(float *A, float *B, float *C, int N) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            register float temp asm("f0");
            temp = 0.0f;
            for (int k = 0; k < N; k++) {
                asm volatile (
                    "flw ft1, %1\n"
                    "flw ft2, %2\n"
                    "fmul.s ft3, ft1, ft2\n"
                    "fadd.s %0, %0, ft3\n"
                    : "+f"(temp)
                    : "m"(A[i * N + k]), "m"(B[k * N + j])
                    : "ft1", "ft2", "ft3"
                );
            }
            C[i * N + j] = temp;
        }
    }
}
```

Here:

- `temp` is explicitly tied to an FPU register (`f0`).

- Intermediate products and sums remain in FPU registers throughout the inner loop.
- Memory is only accessed to fetch the matrix elements and to store the final result.

In a more realistic implementation, the exact register names and constraints would depend on the compiler and toolchain, but the principle is to keep as many hot values as possible in the FPU register file.

8 When This Technique Works Well

Using FPU registers for temporary storage can be effective when:

- There is enough FPU register capacity to hold critical temporaries without frequent spilling.
- The ISA provides straightforward instructions to move data between integer and FPU registers.
- The workload runs in a setting where context-switch overhead is not the dominant factor.

Good candidates include:

- Linear algebra operations (matrix multiplication, dot products).
- Digital signal processing kernels.
- Graphics shaders and numerical simulation kernels.

9 Limitations and Edge Cases

This approach is less beneficial or even harmful when:

- The code is
- The kernel already experiences high register pressure; adding more live values in FPU registers simply moves the spilling problem.
- Portability and maintainability are more important than squeezing out every last percent of performance.

In such cases, relying on the compiler to manage register allocation, or using higher-level optimizations (blocking, tiling, algorithmic changes), may provide better overall gains.

10 Conclusion

FPU registers are extremely fast storage locations intended for floating-point computations, but they can also be used as temporary storage for data that will be reused in the near future. Doing so can reduce load/store instructions and improve performance in carefully chosen kernels, especially on architectures with rich FP/SIMD register files such as ARM with NEON, Intel with SSE/AVX, and RISC-V with the floating-point extension. However, this technique must be applied with an understanding of register pressure, calling conventions, context-switch costs, and portability constraints.

11 Experimental Implementation and Results

To demonstrate the theoretical benefits of using FPU registers for temporary storage, we implemented two versions of a matrix multiplication program (Matrix size 511×511) in RISC-V Assembly.

11.1 Code Approach

11.1.1 Original Code (Naive Approach)

The original code follows a standard mathematical definition of matrix multiplication. It calculates the result one element at a time. For every calculation, it fetches data from memory, multiplies it, adds it to a running total in memory, and repeats. This results in the processor constantly going back and forth to memory (RAM) to get the same data over and over again.

11.1.2 Modified Code (Register Tiling and Unrolling)

The modified code uses a technique called **Register Tiling** (or Blocking). Instead of processing one number at a time, the code attempts to process a "block" of data.

- **Dynamic Tile Selection:** The code checks how many elements are remaining in the row (stored in register `s8`). It effectively asks: "Can I process 32 elements at once? If not, can I process 31?" and so on, down to 1. This is seen in the labels `branch_tile32` through `branch_tile1`.
- **Register Reuse:** In the modified assembly, we load elements of Matrix A into floating-point registers (e.g., `f0`, `f1`) *outside* the inner loop. We then reuse these registers repeatedly against different columns of Matrix B.
- **Reduced Store Operations:** Instead of writing the sum back to memory after every single multiplication, the modified code accumulates the result inside a register and only writes it to memory once the calculation for that tile is finished.

11.2 Performance Comparison

The following table shows the exact performance statistics collected from the simulator for a 511×511 matrix multiplication.

11.3 Analysis of Results: Why is the Modified Code Faster?

The modified code achieved a speedup of approximately **3.65x** (reducing time from 1.56s to 0.43s). Here is the detailed breakdown of why this happened, based on the data above.

11.3.1 1. Drastic Reduction in Memory Access (Loads and Stores)

This is the most important benefit of the modified code.

- **Load Instructions (`ld`):** Dropped from ~ 401 million to ~ 140 million. By keeping the values of Matrix A in FPU registers (like `f0`, `f1`) inside the `k_tile_loop`, the processor does not need to fetch them from memory for every column of Matrix B. It fetches them once per tile and reuses them.

Table 1: Performance Comparison: Original vs. Modified Code

Parameter	Original	Modified
Sim seconds	1.566801	0.429524
Total no of cycles	3,133,601,676	859,047,010
Cycles per instruction	1.110749	1.124131
Instructions per cycle	0.900293	0.889576
Total no of instructions	2,821,161,033	764,187,765
Total no of ld instructions	401,604,107	140,221,986
Total no of store instructions	135,782,926	7,572,515
intAlu	1,747,435,414	336,211,315
intMUL	533,830,487	143,899,171
intdiv	2,611,210	2,611,210
floatcvt	0	133,693,983
Total alu instructions	2,283,877,111	616,415,679
Cache demand (read+write)	478,304,106	147,795,061
Cache hit	469,757,850	138,915,675
Cache miss	8,546,256	8,879,386
Miss rate	0.017868	0.060079

- **Store Instructions:** Dropped massively from ~ 135 million to ~ 7.5 million. The original code updated the result matrix in memory after every multiplication. The modified code calculates the sum in a temporary register and only stores the final value to memory *after* the loop finishes. This saves over 128 million memory write operations.

11.3.2 2. Fewer Total Instructions

The total number of instructions executed dropped by **73%** (from 2.8 billion to 764 million).

- Because we are unrolling the loop (processing blocks of 32, 31, etc.), we perform fewer "jumps" and conditional branch checks (checking if $i < N$).
- The reduction in load and store instructions also contributes directly to this lower total count.

11.3.3 3. Explanation of the "floatcvt" Parameter

You will notice the **floatcvt** (Float Convert) parameter increased from 0 to 133 million.

- The original code likely operated directly on the data format present in memory.
- The modified code explicitly uses instructions like `fcvt.s.w` (convert integer word to single-precision float) and `fcvt.w.s` (convert float to integer word).
- Even though we added 133 million conversion instructions, we saved over 2.3 billion other instructions (loads, stores, and jumps), resulting in a net performance gain.

11.3.4 4. Cache Behavior

An interesting observation is the **Miss Rate**. The miss rate actually increased from 1.7% to 6.0%, but this is not a bad thing in this context.

- The *total number* of cache accesses ("Cache demand") dropped by nearly 330 million.
- Because the processor accesses the cache much less frequently, the "easy" hits (repeatedly reading the same variable from memory) were removed because those variables are now sitting in registers.
- Therefore, the remaining memory accesses are the "harder" ones (fetching new data), which naturally have a higher miss rate, but the **total system time** is much lower.

12 Validation with Irregular Matrix Dimensions

To ensure that our Modified Code (Register Tiling) is robust and works correctly for any matrix size—not just large square matrices—we performed a second experiment with irregular dimensions.

12.1 Experiment Setup

We multiplied two rectangular matrices with the following dimensions:

- **Matrix A:** 291×413
- **Matrix B:** 413×292
- **Result Matrix C:** 291×292

These dimensions were chosen specifically because they are not powers of 2 and do not fit perfectly into the largest tile size (32). This tests the dynamic tile selection logic (e.g., `branch_tile`) to ensure it correctly handles the "edges" of the matrix.

12.2 Performance Statistics

The table below compares the simulation results for this specific configuration. Note that "Cache Hit/Miss" data is omitted as it was not available in this specific dataset, but "Memory Demand" is calculated based on the total load and store instructions issued by the CPU.

12.3 Analysis of Irregular Matrix Results

12.3.1 1. Robustness of the Tiling Logic

The modified code successfully handled the irregular dimensions 291 and 292. The assembly code's dynamic branching logic (checking register `s8` for remaining elements) allowed it to use large tiles (size 32) for the bulk of the work and automatically switch to smaller tiles for the remaining edges. The performance gain remained consistent at approximately **3.7x speedup**.

Table 2: Performance Comparison for Irregular Matrix ($291 \times 413 \times 292$)

Parameter	Original Code	Modified Code
Sim seconds	0.415746	0.112404
Total no of cycles	831,492,355	224,807,277
Total no of instructions	744,265,219	197,989,799
Memory Demand (Ld + St)	141,903,459	38,797,490
Total no of ld instructions	105,886,880	36,947,831
Total no of store instructions	36,016,579	1,849,659
intAlu	461,331,189	86,220,242
intMUL	140,411,883	37,310,856
floatcvt	0	35,213,619
CPI (Cycles Per Instruction)	1.117199	1.135449
IPC (Instructions Per Cycle)	0.895096	0.880709

12.3.2 2. Massive Reduction in Memory Traffic

The "Memory Demand" row highlights the efficiency of the register blocking strategy:

- **Total Memory Accesses:** Dropped from ~ 141 million to ~ 38 million.
- **Store Operations:** The most dramatic improvement was in store instructions, which dropped by **94%** (36 million $\rightarrow$ 1.8 million).

This proves that even with irregular sizes, the strategy of accumulating sums in FPU registers before writing to memory drastically reduces the pressure on the memory subsystem.

12.3.3 3. The Cost of Conversion

Similar to the square matrix experiment, the `floatcvt` (floating-point conversion) instructions increased from 0 to ~ 35 million. However, this cost was negligible compared to the savings in memory access and Integer ALU operations (which dropped by over 375 million instructions), resulting in a significantly faster total execution time.

13 Decimation-in-Time (DIT) FFT Algorithm

The Decimation-in-Time (DIT) Fast Fourier Transform (FFT) is a highly efficient algorithm for computing the Discrete Fourier Transform (DFT). It operates on the principle of divide-and-conquer, significantly reducing the computational complexity from $O(N^2)$ required by a direct DFT implementation to $O(N \log_2 N)$.

The radix-2 DIT algorithm works by recursively decomposing an N -point sequence into two $N/2$ -point subsequences: one containing the even-indexed samples and the other containing the odd-indexed samples. This decomposition continues until 2-point DFTs are reached.

13.1 Algorithm Structure

The execution of an N -point DIT FFT generally involves three main stages:

1. **Bit-Reversal Permutation:** The input time-domain sequence $x(n)$ must be re-ordered before computation begins. The index n is written in binary, the bits are reversed, and this new value becomes the new index. As seen in Figure ??, the input order for $N = 8$ is $x(0), x(4), x(2), x(6), x(1), x(5), x(3), x(7)$.
2. **Butterfly Operations:** The core computational element of the FFT is the "butterfly." It takes two complex inputs (A and B) and a complex "twiddle factor" (phase rotation factor, $W_N^k = e^{-j2\pi k/N}$), and produces two outputs (A' and B'). The basic DIT butterfly equations are:

$$A' = A + W_N^k \cdot B \quad (1)$$

$$B' = A - W_N^k \cdot B \quad (2)$$

3. **Stages:** For an N -point FFT, there are $\log_2 N$ stages.

- **Stage 1:** Combines adjacent pairs using W_N^0 .
- **Stage 2:** Combines pairs separated by 2, using W_N^0 and $W_N^{N/4}$.
- **Subsequent Stages:** The distance between combined pairs and the number of unique twiddle factors doubles with each stage.

The complete signal flow graph for an 8-point DIT FFT, illustrating the bit-reversed inputs, the three computational stages, and the naturally ordered frequency-domain outputs $X(k)$, is shown below in Figure ??.

13.2 RISC-V Assembly Implementation

The following RISC-V assembly code provides a highly optimized implementation of the 8-point FFT structure shown in Figure ??.

This implementation utilizes a "fully unrolled" strategy optimized for the RISC-V floating-point register file. By loading all 16 input data values (complex real and imaginary parts) and necessary twiddle factors into registers at the start, the implementation eliminates intermediate memory load/store operations between stages, significantly reducing memory latency constraints.

The code below shows the core unrolled butterfly operations after data loading. Registers $f0 - f15$ hold data, $f24 - f31$ hold twiddles, and $f16 - f19$ are temporaries.

Listing 1: Optimized RISC-V Assembly for 8-Point DIT FFT Core Stages

```
# --- STAGE 1 ---
# Butterfly 1 (x0, x4) using W^0
fmul.s f16, f2, f24 ; T_real = B_real * W_real
fmul.s f17, f3, f25 ; T_imag = B_imag * W_imag
fsub.s f16, f16, f17 ; Final T_real
fmul.s f17, f3, f24 ; B_imag * W_real
fmul.s f18, f2, f25 ; B_real * W_imag
fadd.s f17, f17, f18 ; Final T_imag

fsub.s f2, f0, f16 ; B' = A - T
fadd.s f0, f0, f16 ; A' = A + T
fsub.s f3, f1, f17
```

```

fadd.s f1, f1, f17

# ... (Similar blocks for x2/x6, x1/x5, x3/x7) ...

# --- STAGE 2 ---
# Butterfly Group 1 (using W^0)
fmul.s f16, f4, f24
fmul.s f17, f5, f25
fsub.s f16, f16, f17
fmul.s f17, f5, f24
fmul.s f18, f4, f25
fadd.s f17, f17, f18
fsub.s f4, f0, f16
fadd.s f0, f0, f16
fsub.s f5, f1, f17
fadd.s f1, f1, f17

# Butterfly Group 2 (using W^2)
fmul.s f16, f6, f28
fmul.s f17, f7, f29
fsub.s f16, f16, f17
fmul.s f17, f7, f28
fmul.s f18, f6, f29
fadd.s f17, f17, f18
fsub.s f6, f2, f16
fadd.s f2, f2, f16
fsub.s f7, f3, f17
fadd.s f3, f3, f17

# --- STAGE 3 (Final mixing using W^0, W^1, W^2, W^3) ---
# ... (Unrolled butterflies using f24-f31 twiddles) ...

```

13.3 Experimentation on different configuration

article booktabs caption float

Table 3: Performance Comparison: Baseline vs. Optimized Implementation (gem5 Simulation) cache access latency = 3, FPU access latency = 2

Parameter	Baseline (Normal)	Optimized (Assembly)
<i>General Performance</i>		
Simulation Time (seconds)	0.012026	0.004412
Total Cycles	24,052,817	8,823,027
Total Instructions	21,671,940	6,252,741
Cycles Per Instruction (CPI)	1.1099	1.4111
Instructions Per Cycle (IPC)	0.9010	0.7087
<i>Memory Access Statistics</i>		
Total Memory Accesses (Ld+St)	4,130,015	1,220,015
Load Instructions	3,050,009	1,100,009
Store Instructions	1,080,006	120,006
<i>ALU Statistics</i>		
Integer ALU Ops	13,461,825	2,862,226
Integer Multiply	4,000,100	1,080,500
Integer Divide	80,000	80,000
Float Convert	0	1,010,000
Total ALU Instructions	17,541,925	5,032,726
<i>Cache Performance (L1 Data)</i>		
Cache Demand (Read+Write)	3,624,942	1,220,122
Cache Hits	3,613,226	1,209,401
Cache Misses	11,716	10,721
Miss Rate	0.32%	0.88%

Note: Configuration set with L1 Cache Latency = 3 cycles, FPU Access Latency = 2 cycles.

Table 4: Performance Comparison: Baseline vs. Optimized Implementation (L1 Cache Latency=5, FPU Latency=2)

Parameter	Baseline	Optimized (Assembly)
<i>General Performance</i>		
Simulation Time (seconds)	0.012970	0.004411
Total Cycles	25,939,711	8,822,433
Total Instructions	21,671,940	6,252,741
Cycles Per Instruction (CPI)	1.1969	1.4110
Instructions Per Cycle (IPC)	0.8355	0.7087
<i>Memory Access Statistics</i>		
Total Memory Accesses (Ld+St)	4,130,015	1,220,015
Load Instructions	3,050,009	1,100,009
Store Instructions	1,080,006	120,006
<i>ALU Statistics</i>		
Integer ALU Ops	13,461,825	2,862,226
Integer Multiply	4,000,100	1,080,500
Integer Divide	80,000	80,000
Float Convert	0	1,010,000
Total ALU Instructions	17,541,925	5,032,726
<i>Cache Performance (L1 Data)</i>		
Cache Demand (Read+Write)	4,053,371	1,220,122
Cache Hits	4,042,695	1,209,401
Cache Misses	10,676	10,721
Miss Rate	0.26%	0.88%

Table 5: Performance Comparison: Baseline vs. Optimized Implementation (L1 Cache Latency=6, FPU Latency=2)

Parameter	Baseline	Optimized (Assembly)
<i>General Performance</i>		
Simulation Time (seconds)	0.012973	0.004412
Total Cycles	25,946,450	8,823,027
Total Instructions	21,671,940	6,252,741
Cycles Per Instruction (CPI)	1.1972	1.4111
Instructions Per Cycle (IPC)	0.8353	0.7087
<i>Memory Access Statistics</i>		
Total Memory Accesses (Ld+St)	4,130,015	1,220,015
Load Instructions	3,050,009	1,100,009
Store Instructions	1,080,006	120,006
<i>ALU Statistics</i>		
Integer ALU Ops	13,461,825	2,862,226
Integer Multiply	4,000,100	1,080,500
Integer Divide	80,000	80,000
Float Convert	0	1,010,000
Total ALU Instructions	17,541,925	5,032,726
<i>Cache Performance (L1 Data)</i>		
Cache Demand (Read+Write)	4,050,217	1,220,122
Cache Hits	4,039,528	1,209,401
Cache Misses	10,689	10,721
Miss Rate	0.26%	0.88%

Table 6: Performance Comparison: Baseline vs. Optimized Implementation (L1 Cache Latency=6, FPU Latency=3)

Parameter	Baseline	Optimized
<i>General Performance</i>		
Simulation Time (seconds)	0.012973	0.004412
Total Cycles	25,946,450	8,823,209
Total Instructions	21,671,940	6,252,741
Cycles Per Instruction (CPI)	1.1972	1.4111
Instructions Per Cycle (IPC)	0.8353	0.7087
<i>Memory Access Statistics</i>		
Total Memory Accesses (Ld+St)	4,130,015	1,220,015
Load Instructions	3,050,009	1,100,009
Store Instructions	1,080,006	120,006
<i>ALU Statistics</i>		
Integer ALU Ops	13,461,825	2,862,226
Integer Multiply	4,000,100	1,080,500
Integer Divide	80,000	80,000
Float Convert	0	1,010,000
Total ALU Instructions	17,541,925	5,032,726
<i>Cache Performance (L1 Data)</i>		
Cache Demand (Read+Write)	4,050,217	1,220,122
Cache Hits	4,039,528	1,209,401
Cache Misses	10,689	10,721
Miss Rate	0.26%	0.88%

Table 7: Performance Comparison: Baseline vs. Optimized Implementation (L1 Cache Latency=6, FPU Latency=4)

Parameter	Baseline	Optimized (Assembly)
<i>General Performance</i>		
Simulation Time (seconds)	0.012973	0.004412
Total Cycles	25,946,450	8,823,959
Total Instructions	21,671,940	6,252,741
Cycles Per Instruction (CPI)	1.1972	1.4112
Instructions Per Cycle (IPC)	0.8353	0.7086
<i>Memory Access Statistics</i>		
Total Memory Accesses (Ld+St)	4,130,015	1,220,015
Load Instructions	3,050,009	1,100,009
Store Instructions	1,080,006	120,006
<i>ALU Statistics</i>		
Integer ALU Ops	13,461,825	2,862,226
Integer Multiply	4,000,100	1,080,500
Integer Divide	80,000	80,000
Float Convert	0	1,010,000
Total ALU Instructions	17,541,925	5,032,726
<i>Cache Performance (L1 Data)</i>		
Cache Demand (Read+Write)	4,050,217	1,220,122
Cache Hits	4,039,528	1,209,401
Cache Misses	10,689	10,721
Miss Rate	0.26%	0.88%